

CKKS 奇偶拆分/重构操作分析

目录

1 分析对象与假设	3
2 噪声分析	4
2.1 噪声模型	4
2.2 automorphism 之后的误差	4
2.3 偶分支的误差	4
2.4 奇分支（带 Y ）的误差	5
2.5 归一化奇分支的误差	6
2.6 一个非常关键的好消息：正向拆分后立刻反向合并时，误差会精确抵消	6
2.6.1 情形 A：保留 Y 因子	6
2.6.2 情形 B：归一化奇分支	7
2.7 噪声层面的结论	7
3 复杂度分析	7
3.1 单次正向拆分的操作计数	8
3.1.1 版本 A：保留 Y 因子	8
3.1.2 版本 B：归一化奇分支	8
3.2 单次反向重构的操作计数	8
3.2.1 从保留 Y 的两支重构	8
3.2.2 从归一化两支重构	8
3.3 哪一步最贵	9
3.4 渐近复杂度	9
3.5 level / multiplicative depth 消耗	10
3.5.1 代数层面	10
3.5.2 实现层面	10
4 如果递归地一直拆分，是否还划算	10
4.1 完全递归到叶子时的总节点数	10
4.2 总运行时间的主导项	11
4.3 内存开销	11
4.4 递归使用时的噪声	11

5	是否 “可以接受”	11
5.1	对“单次拆分 / 单次重构”的判断	12
5.1.1	噪声	12
5.1.2	复杂度	12
5.1.3	结论	12
5.2	对“递归若干层”的判断	12
5.3	对“完整递归到叶子，拿它当加密 FFT 树”的判断	12
6	最终明确结论	13

可以。下面我不给你写实现细节，而是从 **CKKS 语义层** 严谨分析这套“奇偶拆分 / 重构”操作的：

1. 噪声与精度损失
2. 时间复杂度与层级消耗
3. 递归反复使用时是否还能接受

最后我会给一个明确结论。

1 分析对象与假设

我们分析的是下面这套操作。

设

$$P(Y) = \sum_{j=0}^{2n-1} p_j Y^j, \quad Y = X^k, \quad k = \frac{N}{2n},$$

输入密文

$$\mathbf{ct}_P \text{ rep } P(X^k).$$

正向拆分定义为

$$\mathbf{ct}_- := \text{Auto}_{1+2n}(\mathbf{ct}_P),$$

$$\mathbf{ct}_{\text{even}}^\uparrow := \frac{1}{2}(\mathbf{ct}_P + \mathbf{ct}_-),$$

$$\mathbf{ct}_{\text{odd},Y}^\uparrow := \frac{1}{2}(\mathbf{ct}_P - \mathbf{ct}_-),$$

以及可选归一化

$$\mathbf{ct}_{\text{odd}}^\uparrow := (-X^{N-k}) \cdot \mathbf{ct}_{\text{odd},Y}^\uparrow.$$

反向重构为：

- 若保留 Y 因子，则

$$\mathbf{ct}'_P = \mathbf{ct}_{\text{even}}^\uparrow + \mathbf{ct}_{\text{odd},Y}^\uparrow.$$

- 若去掉了 Y 因子，则

$$\mathbf{ct}'_P = \mathbf{ct}_{\text{even}}^\uparrow + X^k \cdot \mathbf{ct}_{\text{odd}}^\uparrow.$$

这些操作在 CKKS 里都属于标准支持的类别：automorphism/Galois、加减、明文-密文乘法。SEAL 明确说明在多项式视角下，Galois automorphism 会把 $\text{Enc}(\text{plain}(x))$ 变成 $\text{Enc}(\text{plain}(x^p))$ ；OpenFHE 和 Lattigo 也都直接提供 automorphism / Galois-key 与 ciphertext-plaintext multiplication 接口。([GitHub][1])

2 噪声分析

2.1 噪声模型

设输入密文 $\mathbf{ct}_P$ 解密后对应的明文多项式为

$$m_P(X) = \Delta P(X^k) + \nu(X),$$

其中:

- Δ 是当前 scale;
- $\nu(X)$ 是输入密文已有的“总误差项”，包含编码舍入误差、加密噪声，以及此前计算积累的近似误差。CKKS 本身就是近似算术方案，rescale 会把噪声和模数一起大致除以 Δ ，但也会引入新的舍入误差。([IACR Eprint Archive][2])

为了写清楚，我们把分析统一放在“消息多项式”层面。

2.2 automorphism 之后的误差

对 $\mathbf{ct}_P$ 做

$$\mathbf{ct}_- := \text{Auto}_{1+2n}(\mathbf{ct}_P),$$

由于

$$\sigma_{1+2n}(X^k) = -X^k,$$

所以消息变为 $P(-X^k)$ 。但 automorphism 在 RLWE/CKKS 实现里并不是“纯置换”，它还需要一次 Galois key switching，因此会注入新的 key-switch 误差。SEAL 文档明确说明应用 Galois automorphism 需要 Galois keys；Lattigo 文档也明确把 GaloisKey 描述为 enabling automorphism $X^i \mapsto X^{i \cdot \text{galEl}}$ 。([GitHub][1])

于是可写成

$$m_-(X) = \Delta P(-X^k) + \sigma(\nu)(X) + \nu_{\text{ks}}(X),$$

其中:

- $\sigma(\nu)$ 是原误差经过 automorphism 后的像;
- ν_{ks} 是这一轮 Galois key switching 新引入的误差。

如果我们用一种对“带符号置换”不变的范数来衡量误差，比如系数 ℓ_∞ 范数，那么

$$|\sigma(\nu)| = |\nu|.$$

2.3 偶分支的误差

偶分支定义为

$$\mathbf{ct}_{\text{even}}^\uparrow := \frac{1}{2}(\mathbf{ct}_P + \mathbf{ct}_-).$$

对应消息为

$$m_{\text{even}}(X) = \frac{1}{2}(m_P(X) + m_-(X)).$$

代入展开：

$$m_{\text{even}}(X) = \frac{1}{2} \left(\Delta P(X^k) + \nu + \Delta P(-X^k) + \sigma(\nu) + \nu_{\text{ks}} \right).$$

利用

$$\frac{P(X^k) + P(-X^k)}{2} = \text{Even}(X^{2k}),$$

得到

$$m_{\text{even}}(X) = \Delta \text{Even}(X^{2k}) + \frac{\nu + \sigma(\nu) + \nu_{\text{ks}}}{2}.$$

所以偶分支的误差项为

$$\nu_{\text{even}} = \frac{\nu + \sigma(\nu) + \nu_{\text{ks}}}{2}.$$

于是有上界

$$|\nu_{\text{even}}| \leq \frac{|\nu| + |\sigma(\nu)| + |\nu_{\text{ks}}|}{2} = |\nu| + \frac{1}{2}|\nu_{\text{ks}}|.$$

这是一个很好的结果：偶分支的误差量级仍与输入误差同阶，只多了半份 automorphism 的 key-switch 误差。

2.4 奇分支（带 Y ）的误差

奇分支定义为

$$\mathbf{ct}_{\text{odd}, Y}^{\uparrow} := \frac{1}{2}(\mathbf{ct}_P - \mathbf{ct}_-).$$

对应消息为

$$m_{\text{odd}, Y}(X) = \frac{1}{2}(m_P(X) - m_-(X)).$$

代入展开：

$$m_{\text{odd}, Y}(X) = \frac{1}{2} \left(\Delta P(X^k) + \nu - \Delta P(-X^k) - \sigma(\nu) - \nu_{\text{ks}} \right).$$

利用

$$\frac{P(X^k) - P(-X^k)}{2} = X^k \text{Odd}(X^{2k}),$$

得到

$$m_{\text{odd}, Y}(X) = \Delta X^k \text{Odd}(X^{2k}) + \frac{\nu - \sigma(\nu) - \nu_{\text{ks}}}{2}.$$

所以奇分支（带 Y ）的误差项为

$$\nu_{\text{odd}, Y} = \frac{\nu - \sigma(\nu) - \nu_{\text{ks}}}{2}.$$

同样有

$$|\nu_{\text{odd}, Y}| \leq |\nu| + \frac{1}{2}|\nu_{\text{ks}}|.$$

因此：

> 偶分支和奇分支的噪声量级是对称的。

2.5 归一化奇分支的误差

若进一步定义

$$\mathbf{ct}_{\text{odd}}^\uparrow := (-X^{N-k}) \cdot \mathbf{ct}_{\text{odd},Y}^\uparrow,$$

因为

$$(-X^{N-k}) \cdot X^k = 1,$$

所以消息变成

$$m_{\text{odd}}(X) = \Delta \text{Odd}(X^{2k}) + (-X^{N-k}) \cdot \nu_{\text{odd},Y}(X).$$

乘上单项式 $(-X^{N-k})$ 本质上只是系数的带符号置换，因此在系数 ℓ_∞ 这类范数下不放大误差：

$$|(-X^{N-k})\nu_{\text{odd},Y}| = |\nu_{\text{odd},Y}|.$$

因此在“理想的环境实现”里，

$$|\nu_{\text{odd}}| \leq |\nu| + \frac{1}{2}|\nu_{\text{ks}}|.$$

如果具体库把这一步实现成 generic plaintext multiplication，并在乘后做 rescale，那么还会额外出现一个标准的 rescale rounding 误差 ρ_{pm} 。CKKS 的 rescaling 会把噪声和模数一起大致除以 Δ ，但会略微增加 worst-case noise-to-modulus ratio。([Massachusetts Institute of Technology][3]) 所以更保守地写：

$$|\nu_{\text{odd}}| \leq |\nu| + \frac{1}{2}|\nu_{\text{ks}}| + \rho_{\text{pm}}.$$

2.6 一个非常关键的好消息：正向拆分后立刻反向合并时，误差会精确抵消

这是这套构造最漂亮的性质。

2.6.1 情形 A：保留 Y 因子

若你从同一个 $\mathbf{ct}_P$ 出发构造了

$$\mathbf{ct}_{\text{even}}^\uparrow = \frac{1}{2}(\mathbf{ct}_P + \mathbf{ct}_-), \quad \mathbf{ct}_{\text{odd},Y}^\uparrow = \frac{1}{2}(\mathbf{ct}_P - \mathbf{ct}_-),$$

那么直接相加：

$$\mathbf{ct}_{\text{even}}^\uparrow + \mathbf{ct}_{\text{odd},Y}^\uparrow = \frac{1}{2}(\mathbf{ct}_P + \mathbf{ct}_-) + \frac{1}{2}(\mathbf{ct}_P - \mathbf{ct}_-) = \mathbf{ct}_P.$$

这不是近似，而是代数上就是同一个密文对象表达式。

在消息层面也同样成立：

$$\nu_{\text{even}} + \nu_{\text{odd},Y} = \frac{\nu + \sigma(\nu) + \nu_{\text{ks}}}{2} + \frac{\nu - \sigma(\nu) - \nu_{\text{ks}}}{2} = \nu.$$

所以：

> 如果你做完 split 之后立刻 merge，那么 automorphism/key-switch 引入的新误差会在两支相加时完全抵消。

这说明：

- 这套 split / merge 不是一个“本质上有损”的变换；
- 额外误差主要只会在你对两个分支做了后续不同计算、或库在中间乘 $1/2$ / rescale 时加入了新的舍入后，才会真正留下来。

2.6.2 情形 B：归一化奇分支

如果你先做了

$$\mathbf{ct}_{\text{odd}}^{\uparrow} = (-X^{N-k})\mathbf{ct}_{\text{odd},Y}^{\uparrow},$$

再在 merge 时乘回 X^k ：

$$X^k \mathbf{ct}_{\text{odd}}^{\uparrow} = X^k (-X^{N-k})\mathbf{ct}_{\text{odd},Y}^{\uparrow} = \mathbf{ct}_{\text{odd},Y}^{\uparrow},$$

于是仍有

$$\mathbf{ct}_{\text{even}}^{\uparrow} + X^k \mathbf{ct}_{\text{odd}}^{\uparrow} = \mathbf{ct}_P.$$

因此，归一化版本在代数层面也是可逆且不额外增噪的；实现里真正可能引入的只是 generic plaintext multiplication / rescale 的舍入误差。

2.7 噪声层面的结论

单次正向拆分的每个分支，其误差规模大约是

$$|\nu| + \frac{1}{2}|\nu_{\text{ks}}|$$

再加上可能存在的少量 plain-mult / rescale rounding。

这比一次 ciphertext-ciphertext multiplication 温和得多，因为：

- 没有 ct-ct 乘法；
- 只有一次 automorphism（本质上是一次 key-switch）；
- 乘到密文上的明文因子都是

$$\frac{1}{2}, \quad X^k, \quad -X^{N-k},$$

它们的数值规模非常小，数值稳定性很好。CKKS 文献和教程都强调：乘法引起的误差增长依赖于明文大小；小系数的 public multiplication 在数值上更稳定。([Massachusetts Institute of Technology][4])

所以从噪声/精度角度看：

> 单次拆分是完全可以接受的。> 它的主要噪声来源只是“一次 automorphism 的 key-switch”，而不是深度乘法链。

3 复杂度分析

下面分两层看：

1. 单次 split / merge 的操作计数
2. 如果递归地一直拆下去，总复杂度会怎样

3.1 单次正向拆分的操作计数

3.1.1 版本 A：保留 Y 因子

$$\mathbf{ct}_P \longrightarrow (\mathbf{ct}_{\text{even}}^\uparrow, \mathbf{ct}_{\text{odd},Y}^\uparrow)$$

所需操作是：

- 1 次 automorphism
- 1 次加法
- 1 次减法
- 2 次明文-密文乘法（乘 $1/2$ ）

所以可以记为

$$C_{\text{split,keepY}} = C_{\text{auto}} + 2C_{\text{add}} + 2C_{\text{pmul}}.$$

3.1.2 版本 B：归一化奇分支

再多一步

$$(-X^{N-k}) \cdot \mathbf{ct}_{\text{odd},Y}^\uparrow,$$

所以

$$C_{\text{split,norm}} = C_{\text{auto}} + 2C_{\text{add}} + 3C_{\text{pmul}}.$$

3.2 单次反向重构的操作计数

3.2.1 从保留 Y 的两支重构

只需

$$\mathbf{ct}_{\text{even}}^\uparrow + \mathbf{ct}_{\text{odd},Y}^\uparrow,$$

所以

$$C_{\text{merge,keepY}} = C_{\text{add}}.$$

3.2.2 从归一化两支重构

需要先乘回 X^k ，再加法：

$$\mathbf{ct}_{\text{even}}^\uparrow + X^k \mathbf{ct}_{\text{odd}}^\uparrow,$$

所以

$$C_{\text{merge,norm}} = C_{\text{pmul}} + C_{\text{add}}.$$

3.3 哪一步最贵

这里结论非常明确：

> 单次 split 的主成本是那一次 automorphism。

原因是：

- 加法/减法通常是最便宜的一类同态操作；
- scalar/plaintext multiplication 通常也远便宜于 key-switch；
- automorphism 在 CKKS/RLWE 实现里本质上要做一次 Galois key switching，而 key switching 是 HE 里著名的重操作。SEAL 明确要求 Galois keys；Lattigo 也明确把 GaloisKey 描述为 enabling automorphism $X^i \mapsto X^{i \cdot \text{galEl}}$ 。而近期实现论文也明确把 rotations/automorphisms 的核心成本归结为 key-switching，并指出当同一密文需要多个 rotations 时可通过 hoisting 降低总运行时间。([GitHub][1])

所以在工程上可以安全地把单次 split 的成本近似看成：

$$\boxed{\text{一次 split 的成本} \approx \text{一次 automorphism 的成本}}$$

其余两次加减和两三次明文乘法只是次要项。

3.4 渐近复杂度

若记：

- 环维数为 N
- 当前模链长度为 L

那么：

- add/sub 大致是线性系数运算，量级可看成

$$O(LN)$$

- scalar/plaintext multiplication 的成本取决于实现：
 - 若是标量 fast-path 或 monomial permutation，可接近 $O(LN)$
 - 若走 generic NTT-based plaintext multiplication，则可接近

$$O(LN \log N)$$

- automorphism 的主成本是 key-switch；而 key-switch 本身由一组高维多项式运算组成，通常是整步里最昂贵的部分。已有实现与硬件论文都把 key-switching 视为计算热点。([IACR Eprint Archive][5])

因此单次 split 的渐近复杂度可以写成：

$$\boxed{O(\text{KeySwitch}(N, L)) + O(LN)}$$

其中第一项绝对主导。

3.5 level / multiplicative depth 消耗

这点要分“代数层面”和“库实现层面”。

3.5.1 代数层面

我们的算法没有任何 ct-ct multiplication，所以：

- automorphism：不增加乘法深度
- add/sub：不增加乘法深度
- monomial multiplication：若被视为 scale-1 的环内乘子，也不需要额外 rescale

因此从纯算法结构看，这个变换是**低深度**的，几乎是 depth-0 的线性变换。

3.5.2 实现层面

实际库如果把 $1/2$ 、 X^k 、 $-X^{N-k}$ 作为 generic plaintext 来乘，则可能需要：

- 做 plaintext encoding
- 更新 scale
- 有时再 rescale 一次

所以真正的“level 消耗”取决于库如何实现 plaintext multiplication。MIT 的 CKKS 教程指出，rescaling 会把噪声和模数一起大致除以 Δ ，但会引入新的 rounding。([Massachusetts Institute of Technology][3])

因此工程上更稳妥的判断是：

> 这套 split / merge 本身不是“深度型”操作；若出现 level 消耗，几乎都来自实现层面对 plaintext multiplication 的处理，而不是来自算法本质。

4 如果递归地一直拆分，是否还划算

这部分很重要，因为单步可行，不代表全递归也划算。

4.1 完全递归到叶子时的总节点数

初始多项式有 $2n$ 个系数。每次拆分把一个“长度 $2m$ ”的多项式拆成两个“长度 m ”的多项式。若一直拆到常数项为止，就得到一棵满二叉树：

- 叶子数： $2n$
- 内部节点数： $2n - 1$

而**每个内部节点都需要做一次 split**，也就是至少一次 automorphism。

因此，完全递归拆分到叶子的总 automorphism 数量是

$$2n - 1.$$

这很关键，因为 automorphism 是整件事里最贵的部分。

4.2 总运行时间的主导项

如果记一次 automorphism 的成本为 C_{auto} ，则完整递归树的主导成本大约是

$$(2n - 1) C_{\text{auto}}.$$

加法和明文乘法的总数也会线性增长，但仍通常是次要项。

因此：

完整递归 FFT 式拆分的总时间，主导项线性正比于 split 节点数 $(2n - 1)$

4.3 内存开销

单次 split 会把 1 个密文变成 2 个密文，因此如果你“整层全部保留”：

- 第 0 层：1 个密文
- 第 1 层：2 个
- 第 2 层：4 个
- ...

如果整棵树都物化出来，存储的密文数量会达到线性级别

$$O(n).$$

这在工程上也不便宜，因为 CKKS 密文对象本身就很大。

当然，若采用深度优先递归并及时释放中间节点，峰值内存可以降很多；但这不改变 automorphism 总数。

4.4 递归使用时的噪声

若只是 split 以后立刻 merge，误差会很好地抵消。但若你在两支上继续做不同计算，再在更深层重组，那么：

- split 时那一次 automorphism 的误差已经进入各支；
- 后续各支的误差会分别增长；
- 之后一般不会再像“立刻 merge”那样精确抵消。

所以递归树越深，噪声仍会逐层积累，只是它的积累方式更接近“每个 split 节点注入一次 key-switch 误差”，而不是“每层做一次 ct-ct multiplication”。

换句话说：

> 递归树的瓶颈更像是“很多次 automorphism / key-switch 的累计成本”，而不是乘法深度本身。

5 是否“可以接受”

现在可以给你一个明确判断。

5.1 对“单次拆分 / 单次重构”的判断

5.1.1 噪声

单次 split 的每个分支的误差规模大致是

$$|\nu| + \frac{1}{2}|\nu_{\text{ks}}|$$

再加上可能很小的 plaintext-multiplication / rescale rounding。这是一个非常温和的增长，因为这里没有 ct-ct multiplication，明文乘子又都是小量或单位模单项式。([Massachusetts Institute of Technology][4])

5.1.2 复杂度

单次 split 的主要成本就是 **一次 automorphism/key-switch**；其余操作都便宜得多。反向 merge 更轻，几乎只是一次加法，或“单项式明文乘法 + 加法”。([GitHub][1])

5.1.3 结论

单次奇偶拆分 / 单次重构是可以接受的。

更强一点说：

- 从**噪声**看，它很健康；
- 从**深度**看，它几乎不消耗乘法深度；
- 从**时间**看，它只贵在一次 automorphism，而这在 CKKS 里完全是正常量级。

5.2 对“递归若干层”的判断

若只做 1~2 层、或者做少量结构化分支，这仍然通常是可接受的，因为：

- 每层只引入一次 automorphism；
- 没有高深度乘法链；
- 分支多项式次数迅速减半。

所以：

做少量层数的递归拆分，通常也是可以接受的。

5.3 对“完整递归到叶子，拿它当加密 FFT 树”的判断

这里结论就没有那么乐观了。

因为完整树需要

$$2n - 1$$

次 split，也就是

$$2n - 1$$

次 automorphism / key-switch。对中大规模参数，这会很快变得昂贵：

- 总时间由大量 key-switch 主导；
- 密文个数膨胀；
- 各分支误差继续累加，尽管没有 ct-ct multiplication 那么糟，但也并不免费。

因此：

把它递归到很深、当作“完整加密 FFT”的基础原语，一般不算特别划算。

它更适合：

- 做一层或少量层数的结构化分裂；
- 作为某个更大算法中的局部子步骤；
- 在你确实需要“多项式系数域的奇偶分支”而不是“槽域 FFT/DFT”时使用。

6 最终明确结论

我给你一个尽量清楚、可直接引用的结论：

结论：这套奇偶拆分 / 重构在 CKKS 上是噪声可接受、复杂度可接受的，前提是把它当作单次或少量层数的局

更具体地：

1. **单次正向拆分是可接受的：**每个分支只额外承受“一次 automorphism 的 key-switch 噪声”的量级，没有 ct-ct multiplication，因此精度损失温和。
2. **单次反向重构是非常可接受的：**若 split 后立刻 merge，理论上新引入的 automorphism 噪声会在两支相加时精确抵消；实际库中只会剩下少量 plaintext multiplication / rescale rounding。
3. **时间上，单次 split 的主成本就是一次 automorphism/key-switch，**其余都是便宜操作。
4. **若完整递归到叶子，则总成本会被 $2n - 1$ 次 automorphism 主导，**此时一般就不再是“很划算”的方案。

所以最终建议是：

可用，但应把它定位为“局部的、多项式域奇偶拆分原语”，而不是“大规模完整加密 FFT”的默认实现方式。

如果你愿意，我下一条可以继续把这个分析再往前推一步，给你写成一个更像论文里的 **Proposition + Cost Model + Final Corollary** 的版本。

参考文献

- [1] <https://github.com/microsoft/SEAL/blob/main/native/src/seal/evaluator.h>
- [2] <https://eprint.iacr.org/2016/421>
- [3] https://www.mit.edu/~linust/files/CKKS_Homomorphic_Encryption_Part_1.pdf
- [4] https://www.mit.edu/~linust/files/CKKS_Homomorphic_Encryption_Part_3.pdf
- [5] <https://eprint.iacr.org/2025/124.pdf>